Computer Vision and Mixed Reality (VARM)

1.º Projeto — Face Detection and Recognition for Augmented Reality

Instituto Superior de Engenharia de Lisboa
Mestrado em Engenharia Informática e Multimédia

Bruno Rodrigues

Índice

1. Introdução *3*

2. Arquitetura e Organização do Código *3*

2.1 main.py — Entry point **3**

2.2 normalizer.py — Pipeline de normalização MPEG-7 ..... **3**

2.3 fisher_recognizer.py — Reconhecedor FisherFaces **4**

2.4 Assets — imagens de augmented reality **4**

3. Pipeline Completo *4*

3.1 Offline — Preparação do dataset **4**

3.2 Online — Runtime **4**

4. Decisões de Desenho e Problemas Encontrados *5*

4.1 LBPH vs FisherFaces **5**

4.2 Threshold para a classe "Desconhecido" **5**

4.3 Dimensão do dataset e classes mínimas **5**

4.4 Sincronização incremental do dataset **5**

4.5 Descoberta dinâmica de classes **6**

5. Augmented Reality — Sobreposição de Objetos Virtuais *6*

6. Resultados *6*

7. Limitações e Trabalho Futuro *7*

8. Como Correr o Projeto *7*

1. Introdução

Este projeto implementa uma aplicação capaz de detetar e reconhecer caras em tempo real a partir de uma webcam. A aplicação deteta qualquer cara presente no frame, tenta identificar a pessoa face a um conjunto de classes conhecidas (treinadas a partir de um dataset), e sobrepõe um elemento virtual (óculos, chapéu, ou um ponto de interrogação) alinhado com a cara detetada, de forma a criar uma experiência de Augmented Reality (AR) simples mas funcional.

Os três pilares do projeto, conforme definidos no enunciado, são: (a) deteção de face em imagens capturadas em tempo real, (b) reconhecimento facial com extração de características e classificação, e (c) combinação de objetos reais e virtuais com alinhamento. Todas as etapas foram implementadas em Python com a biblioteca OpenCV (`opencv-contrib-python`), sem recurso a redes neuronais nem deep learning.

2. Arquitetura e Organização do Código

A aplicação é composta por quatro ficheiros Python, cada um com uma responsabilidade bem delimitada. A separação de responsabilidades permite testar os componentes isoladamente e justificar as decisões algorítmicas de cada fase.

2.1 main.py — Entry point

Responsável pelo parsing de argumentos da linha de comandos (--live, --image, --dataset) e orquestração de alto nível: (1) sincroniza o dataset normalizado com o dataset original em bruto caso haja fotos novas, (2) instância o reconhecedor, (3) treina-o, e (4) dispara um dos três modos de operação (webcam em tempo real, avaliação no test set, ou reconhecimento numa imagem estática).

A função `sync_normalized` implementa uma estratégia de sincronização incremental: percorre todas as fotos em `dataset/` e só executa o pipeline de normalização sobre as que ainda não existem em `dataset_normalized/`. Runs sem fotos novas não têm novos custos computacionais nem aumentam o tempo de execução. O normalizador só é instanciado se for realmente necessário.

2.2 normalizer.py — Pipeline de normalização MPEG-7

A classe `FaceNormalizer` transforma uma foto em bruto numa imagem normalizada segundo a recomendação MPEG-7: monocromática de 46 colunas × 56 linhas, com os olhos esquerdo e direito alinhados horizontalmente nas posições (coluna 16, linha 24) e (coluna 31, linha 24) respetivamente.

O pipeline de normalização é composto por 4 fases:

- Deteção de face — Haar Cascade (`haarcascade_frontalface_default`) seleciona a face de maior área.
- Deteção de olhos — Haar Cascade para olhos, com equalização de histograma e relaxamento progressivo dos parâmetros (scaleFactor, minNeighbors, minSize) até encontrar pelo menos 2 olhos. Tenta primeiro o cascade normal e depois o cascade para olhos com óculos.
- Rotação — calcula o ângulo entre os dois olhos e aplica `warpAffine` com a matriz dada por `getRotationMatrix2D`, usando o ponto médio entre os olhos como centro de rotação.
- Escala e recorte — calcula o fator de escala necessário para a distância entre olhos bater exatamente os 15 pixels alvo (colunas 16 a 31), aplica `cv2.resize`, e recorta a região 46×56 em torno das

posições alvo. Se o recorte cair fora dos limites da imagem escalada, aplica padding com `cv2.copyMakeBorder` usando `BORDER_REPLICATE`.

2.3 fisher_recognizer.py — Reconhecedor FisherFaces

A classe `FisherFaceRecognizer` encapsula o pipeline de reconhecimento com FisherFaces. Carrega as imagens já normalizadas do `dataset_normalized/`, descobrindo dinamicamente os nomes das pessoas a partir das subpastas de `train/` (sem nomes hardcoded). Treina um modelo `cv2.face.FisherFaceRecognizer` com `num_components = num_classes - 1`.

O método `predict` devolve o nome mais provável e uma confiança. Se a confiança ultrapassar o `UNKNOWN_THRESHOLD`, a cara é marcada como "Desconhecido" — esta rejeição é essencial para o requisito da "rejection class" do enunciado.

O método `live` implementa o loop de captura da webcam: para cada frame aplica o Haar Cascade, para cada face detetada chama `predict`, desenha a bounding box e o rótulo, e chama `overlay` para sobrepor o elemento virtual correspondente à pessoa identificada.

2.4 Assets — imagens de augmented reality

Três PNGs com canal alfa servem de objetos virtuais: `glasses.png` (óculos), `hat.png` (chapéu), e `???.png` (ponto de interrogação). O canal alfa é usado para blending por máscara alfa: o overlay aceita imagens RGBA e mistura cada canal de cor com o frame original usando `alpha * overlay + (1-alpha) * frame`.

3. Pipeline Completo

O fluxo de dados da aplicação pode ser visto a dois níveis: offline (preparação de dados) e online (runtime).

3.1 Offline — Preparação do dataset

fotos cruas → dataset/train/<Pessoa>/ → FaceNormalizer → dataset_normalized/train/<Pessoa>/

Este passo é responsável por gerar o conjunto de treino homogéneo que alimenta o FisherFaces. A homogeneidade das imagens (mesmo tamanho, mesmo alinhamento dos olhos) é uma condição necessária para o FisherFaces funcionar — qualquer variação de pose, escala ou inclinação entre amostras degrada drasticamente a base do espaço PCA/LDA.

3.2 Online — Runtime

frame da webcam → CascadeClassifier (deteção de face) → resize 46×56 → FisherFaceRecognizer.predict() → overlay do objeto virtual → cv2.imshow

Note-se que em runtime a imagem da webcam NÃO passa pelo normalizador MPEG-7 completo (não são detetados olhos nem aplicada rotação). A face detetada é simplesmente redimensionada para 46×56 antes de ser dada ao modelo. Esta simplificação é intencional: o custo de correr o pipeline completo a

cada frame tornaria a aplicação não real-time. Para compensar, o dataset de treino é normalizado de forma robusta, e o modelo aprende a tolerar pequenas variações de alinhamento face à base canónica.

4. Decisões de Desenho e Problemas Encontrados

4.1 LBPH vs FisherFaces

Começámos com LBPH (Local Binary Patterns Histograms) por ser o algoritmo mais simples e insensível a iluminação. Com o dataset original (não normalizado) o LBPH atingiu 52% de accuracy, subindo para 65% após afinar o `UNKNOWN_THRESHOLD`. Porém, quando aplicado ao dataset normalizado MPEG-7 o LBPH colapsou para 0%: as imagens 46×56 são demasiado pequenas para gerar histogramas suficientemente discriminativos.

Fizemos então a transição para FisherFaces (PCA + LDA), que é conhecido por tirar partido precisamente de imagens com alinhamento canónico e mesmo tamanho — ou seja, é complementar ao pipeline MPEG-7. Com FisherFaces sobre o dataset normalizado atingimos 80% de accuracy, um salto significativo.

4.2 Threshold para a classe "Desconhecido"

Tanto LBPH como FisherFaces devolvem sempre a classe "mais próxima" — por si só, não sabem dizer "não conheço esta pessoa". A classe de rejeição é implementada via threshold sobre a confiança devolvida. Para FisherFaces a escala de confiança é bastante diferente do LBPH (valores nas centenas/milhares), portanto o valor do threshold é específico do algoritmo e do dataset, e teve de ser afinado experimentalmente apontando a webcam para caras conhecidas e desconhecidas e observando os valores devolvidos.

4.3 Dimensão do dataset e classes mínimas

Um problema encontrado foi que FisherFaces com apenas 2 classes reduz o espaço discriminativo a um único eixo (`num_components = num_classes - 1 = 1`), tornando as classes colapsadas e o modelo incapaz de separar pessoas com aparência parecida. A solução foi garantir pelo menos 3 classes no treino, o que dá ao LDA 2 eixos para trabalhar e melhora substancialmente a separabilidade.

Igualmente importante é ter um número razoável de fotos por pessoa (ordem de 10-25) com variação real de pose, iluminação e expressão. A qualidade da variação importa mais que a quantidade — 15 fotos variadas superam 30 repetitivas.

4.4 Sincronização incremental do dataset

A primeira versão exigia correr `python normalizer.py` à mão sempre que se acrescentavam fotos novas. Era um passo fácil de esquecer e resultava em runs com dataset desatualizado sem aviso. A solução foi integrar o normalizador no arranque do `main.py`, mas de forma incremental: só são normalizadas as fotos que ainda não têm correspondente em `dataset_normalized/`. Runs sem novidades têm custo zero (o `FaceNormalizer` nem sequer é instanciado).

4.5 Descoberta dinâmica de classes

Inicialmente a lista `person_names` estava hardcoded no código. Mudámos para descoberta dinâmica por listagem de subpastas em `train/`, ordenadas alfabeticamente. Adicionar uma nova pessoa ao dataset passou a ser trivial: bastar criar a pasta com o nome e pôr lá as fotos — o código descobre-a automaticamente no próximo run.

5. Augmented Reality — Sobreposição de Objetos Virtuais

O requisito de AR do enunciado especifica pelo menos três objetos virtuais distintos: um para o estudante, um para o professor, e um para "outros" (desconhecidos). Implementámos:

- Óculos (`glasses.png`) — sobrepostos na zona dos olhos quando a cara é reconhecida como Bruno Rodrigues. O retângulo de destino é calculado como sendo a largura total da face e 35% da altura, posicionado a 25% do topo da bounding box.
- Chapéu (`hat.png`) — sobreposto acima da cabeça quando a cara é reconhecida como Pedro M Jorge. A largura é 1.4× a da face (o chapéu é mais largo que a cabeça) e a altura é 90% da face, posicionado acima do topo da bounding box.
- Ponto de interrogação (`???.png`) — sobreposto acima da cabeça para qualquer cara marcada como "Desconhecido". Serve como marcador visual da classe de rejeição.

A sobreposição é implementada pelo método `overlay`, que aceita uma imagem PNG com canal alfa, redimensiona-a para o retângulo de destino, trata o caso do retângulo cair parcialmente fora do frame (crop seguro), e faz blending linear usando o canal alfa como máscara:

```
frame[y:y+h, x:x+w, c] = alpha * overlay[..., c] + (1 - alpha) * frame[y:y+h, x:x+w, c]
```

Esta abordagem satisfaz o requisito "adding with mask" do enunciado sem necessidade de `cv2.add` ou `cv2.addWeighted` (que exigiriam máscaras binárias ou pesos uniformes). O canal alfa permite transições suaves nas bordas dos PNGs.

6. Resultados

A tabela seguinte compara as configurações exploradas durante o desenvolvimento, avaliadas sobre o test set:

Configuração	Accuracy
LBPH + dataset original (sem normalização)	52%
LBPH + dataset original (threshold ajustado)	65%
LBPH + dataset normalizado MPEG-7	0% (threshold desajustado)
FisherFaces + dataset normalizado (sem padding/equalização)	50%
FisherFaces + dataset normalizado (com padding + equalização)	80%

A configuração final da aplicação é FisherFaces sobre `dataset_normalized/` com padding e equalização de histograma no pipeline MPEG-7, que rendeu 80% de accuracy. Em modo live, com 3 classes de treino (Bruno, Pedro, AJ) e uma quarta pessoa como teste de rejeição, observamos que caras desconhecidas

são consistentemente marcadas como "Desconhecido" e as caras treinadas são classificadas corretamente na maioria dos frames, com ocasional flicker entre classes quando a iluminação é desfavorável ou o ângulo facial é extremo.

7. Limitações e Trabalho Futuro

- O pipeline de deteção usa Haar Cascades, que é sensível a ângulos laterais extremos, oclusão parcial, e baixa iluminação. Uma versão futura poderia substituir por DNN (ResNet-SSD disponível no módulo `cv2.dnn`) para deteção mais robusta, mas ao custo de mais latência.
- O reconhecimento não usa tracking temporal entre frames — cada frame é classificado independentemente. Em cenários com iluminação variável isto causa flicker ocasional no rótulo. Uma melhoria trivial seria aplicar suavização temporal (moda das últimas N classificações) para estabilizar a saída.
- O threshold de rejeição é um valor fixo hardcoded. Idealmente seria calibrado automaticamente a partir do histograma de confianças do train set vs um conjunto de validação de caras desconhecidas.
- O dataset é pequeno (ordem de 10-20 imagens por pessoa). Aumentar o dataset e garantir variação forte de pose/iluminação é a intervenção com maior retorno para subir a accuracy.

8. Como Correr o Projeto

Pré-requisitos: Python 3.9+ e a biblioteca opencv-contrib-python.

```
pip install opencv-contrib-python numpy
```

A partir da pasta `p1/`:

```
python main.py --live     # webcam em tempo real com overlays AR

python main.py            # avalia o modelo sobre o test set

python main.py --image bruno.jpg   # reconhece uma única imagem
```

No modo `--live`, a tecla `q` termina a aplicação. Para adicionar uma pessoa nova, criar `dataset/train/<Nome>/` e colocar as fotos lá dentro — no próximo arranque o `main.py` normaliza-as automaticamente e o modelo passa a conhecê-las.

Estrutura de pastas esperada:

```
p1/
├── main.py                 # entry point
├── fisher_recognizer.py    # FisherFaces + overlays AR
├── normalizer.py           # pipeline MPEG-7
├── glasses.png, hat.png, ???.png   # assets AR
├── dataset/                # fotos originais (train/ + test/)
└── dataset_normalized/     # fotos 46×56 alinhadas (gerado)
```